



计算机组成与设计

龚奕利

yiligong@whu.edu.cn

武汉大学计算机学院

Computer Organization and Design

—Course Administration

■ Text:

Computer Organization and Design:

The Hardware/Software Interface,

David A. Patterson and John L. Hennessy,

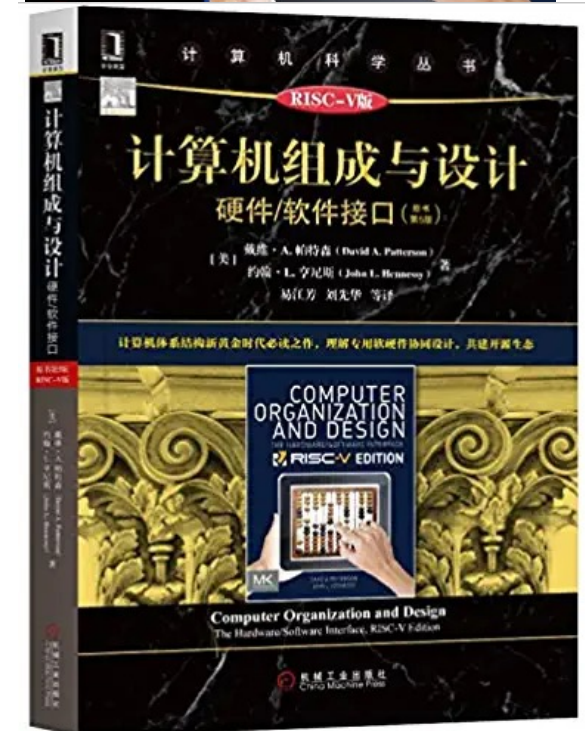
5th Ed., RISC-V Edition.

■ Prerequisite:

- Digital system

■ Grade breakdown

- | | |
|------------------------|-----|
| ■ Class | 10% |
| ■ Homework Assignments | 15% |
| ■ Labs | 25% |
| ■ Final exam | 50% |



Getting Help

- Email: yiligong@whu.edu.cn
- QQ群



- Office hour :
 - 每周四上午9:30-11:30 @计算机学院A515

不讲的内容

- 第1章：1.5, 1.8~1.9
- 第2章：2.15~2.17
- 第3章：3.3, 3.4, 3.6~3.8
- 第4章：4.11~4.13, 4.9和4.10选讲（会讲）
- 第5章：5.6, 5.9~5.14, 5.15选讲（会讲）
- 第6章：不讲

每章的谬误和陷阱自行学习

Chapter 1

Computer Abstractions and Technology

The Computer Revolution

- Progress in computer technology
 - Underpinned by Moore's Law
- Makes novel applications feasible
 - Cell phones
 - Human genome project
 - Search Engines
 - Artificial Intelligence, Deep Learning
 - Self-driving cars
- Computers are pervasive

Classes of Computers

- Personal computers
 - General purpose, variety of software
 - Subject to cost/performance tradeoff
- Server computers
 - Network based
 - High capacity, performance, reliability
 - Range from small servers to building sized

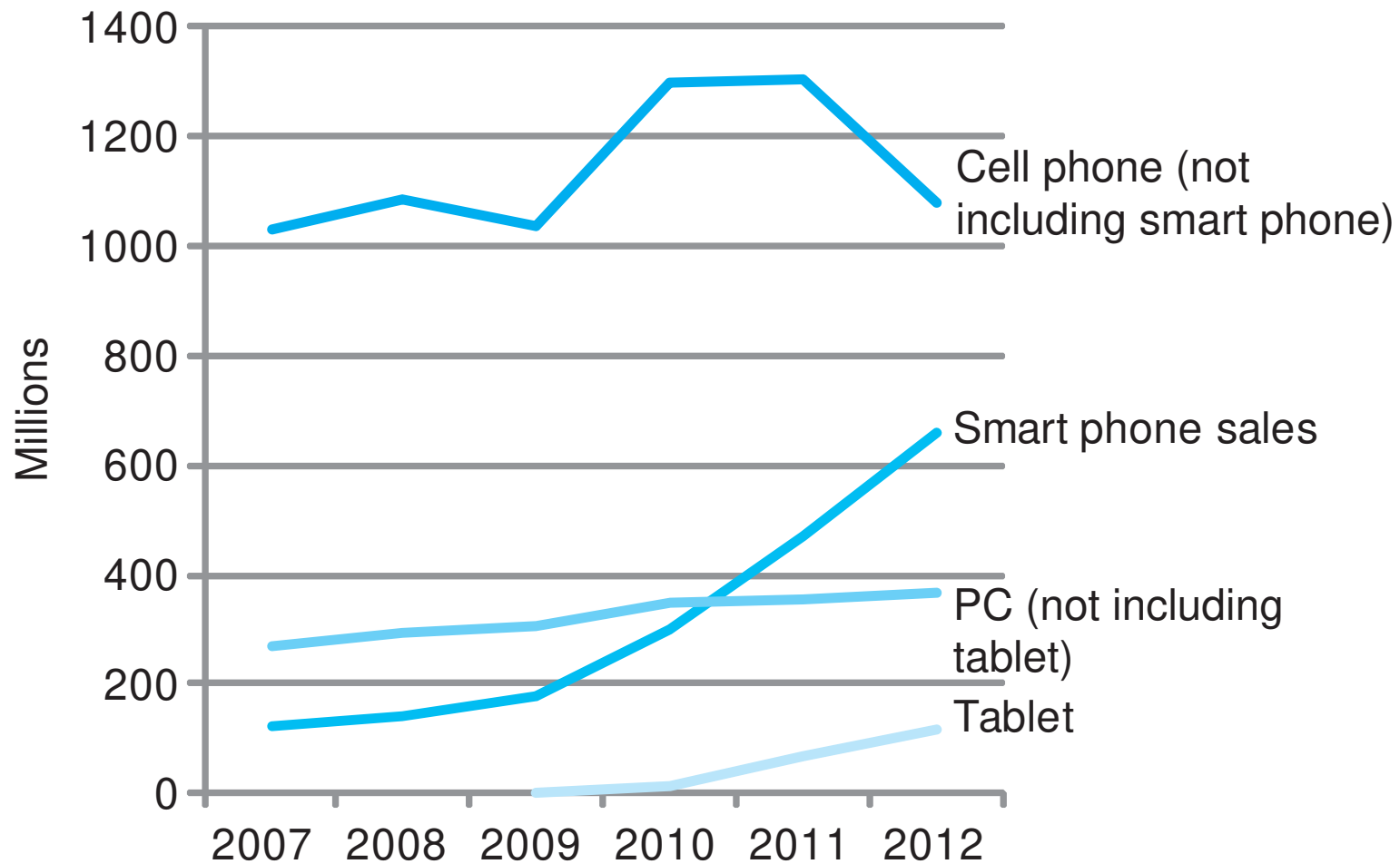
Classes of Computers

- Supercomputers
 - High-end scientific and engineering calculations
 - Highest capability but represent a small fraction of the overall computer market
- Embedded computers
 - Hidden as components of systems
 - Stringent power/performance/cost constraints

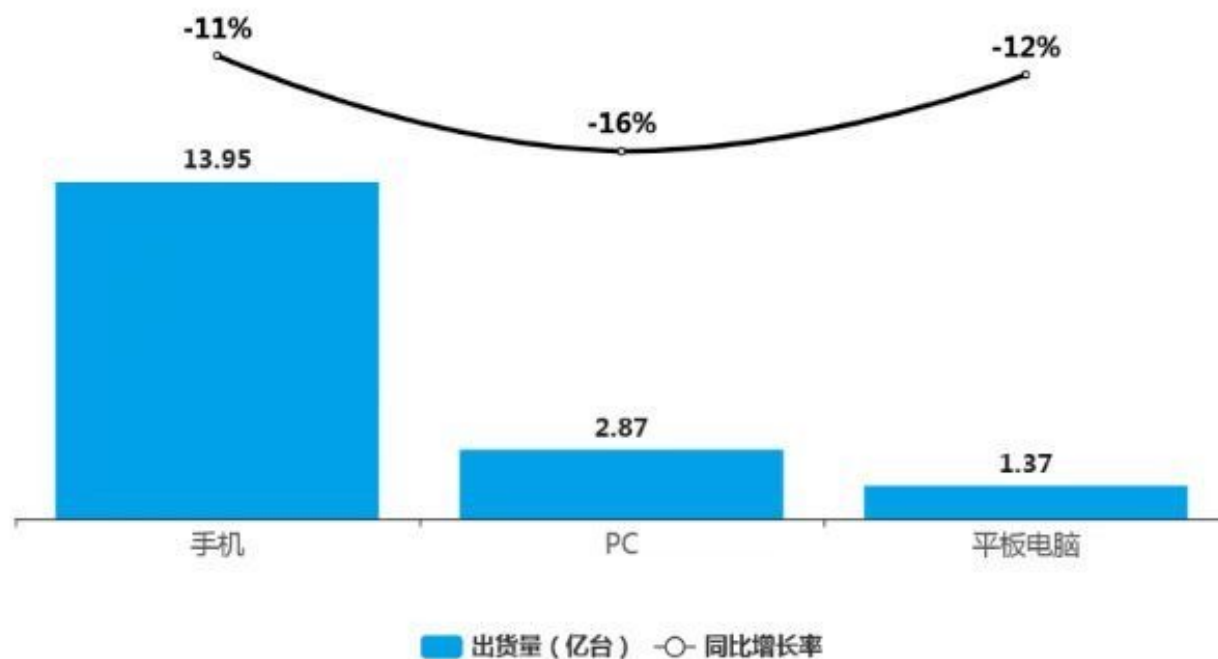
Full Range of Decimal and Binary Values and Names

Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%

The PostPC Era



2022年全球PC平板电脑和手机出货量及增长率



The PostPC Era

- Personal Mobile Device (PMD)
 - Battery operated
 - Connects to the Internet
 - Hundreds of dollars
 - Smart phones, tablets, electronic glasses



The PostPC Era



- Cloud computing
 - Warehouse Scale Computers (WSC)
 - Software as a Service (SaaS)
 - Portion of software run on a PMD and a portion run in the Cloud
 - Amazon and Google

What You Will Learn

- How programs are translated into the machine language
 - And how the hardware executes them
- The hardware/software interface
- What determines program performance
 - And how it can be improved
- How hardware designers improve performance
- What is parallel processing

Understanding Performance

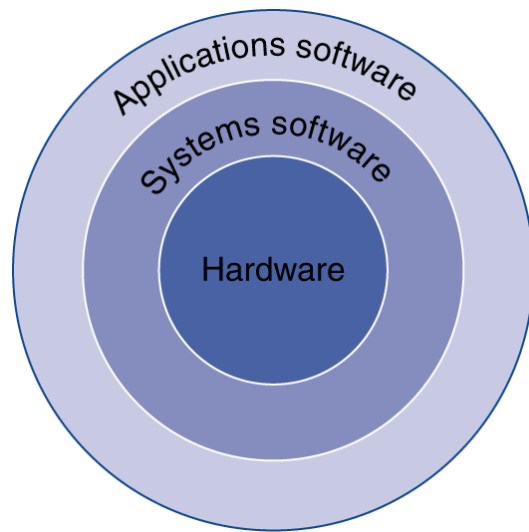
- Algorithm
 - Determines number of operations executed
- Programming language, compiler, architecture
 - Determine number of machine instructions executed per operation
- Processor and memory system
 - Determine how fast instructions are executed
- I/O system (including OS)
 - Determines how fast I/O operations are executed

Eight Great Ideas

- Design for **Moore's Law**
- Use **abstraction** to simplify design
- Make the **common case fast**
- Performance via **parallelism**
- Performance via **pipelining**
- Performance via **prediction**
- **Hierarchy** of memories
- **Dependability** via redundancy



Below Your Program



- Application software
 - Written in high-level language
- System software
 - Compiler: translates HLL code to machine code
 - Operating System: service code
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources
- Hardware
 - Processor, memory, I/O controllers

Levels of Program Code

- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- Assembly language
 - Textual representation of instructions
- Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for RISC-V)

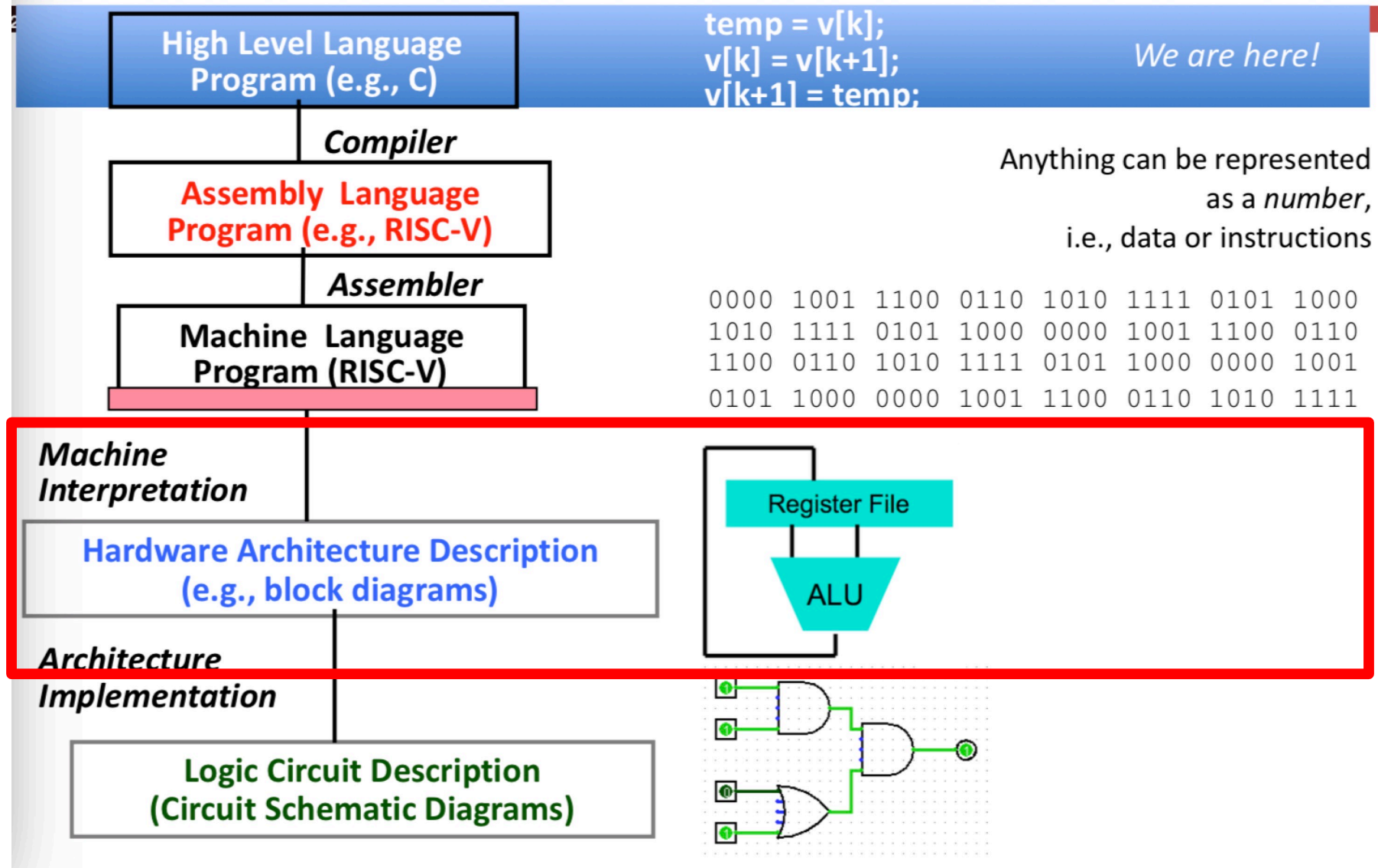
```
swap:
    slli x6, x11, 3
    add  x6, x10, x6
    ld   x5, 0(x6)
    ld   x7, 8(x6)
    sd   x7, 0(x6)
    sd   x5, 8(x6)
    jalr x0, 0(x1)
```

Assembler

Binary machine
language
program
(for RISC-V)

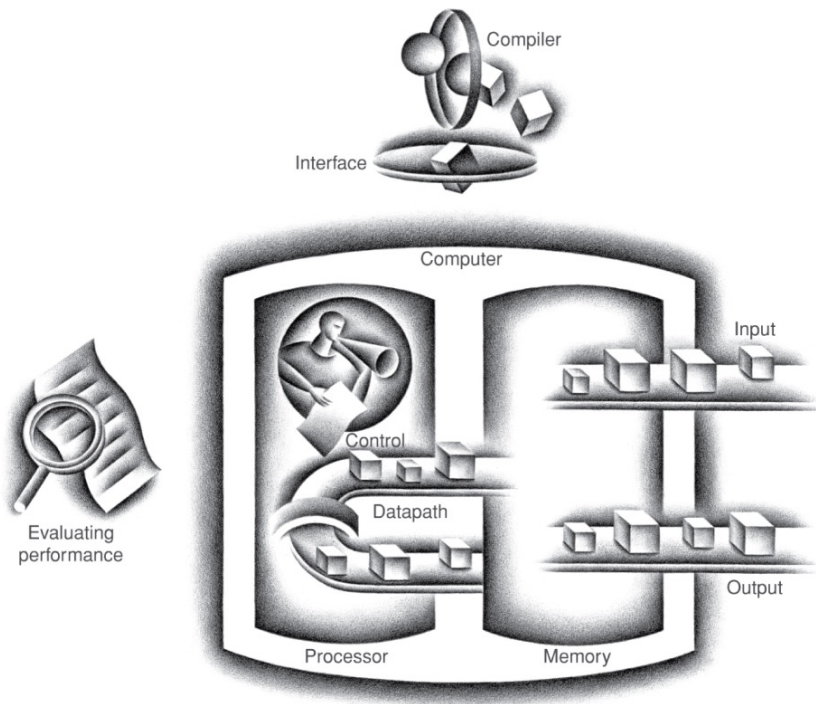
```
00000000001101011001001100010011
000000000011001010000001100110011
000000000000000110011001010000011
000000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000000100000001100111
```

Great Idea: Levels of Representation/Interpretation



Components of a Computer

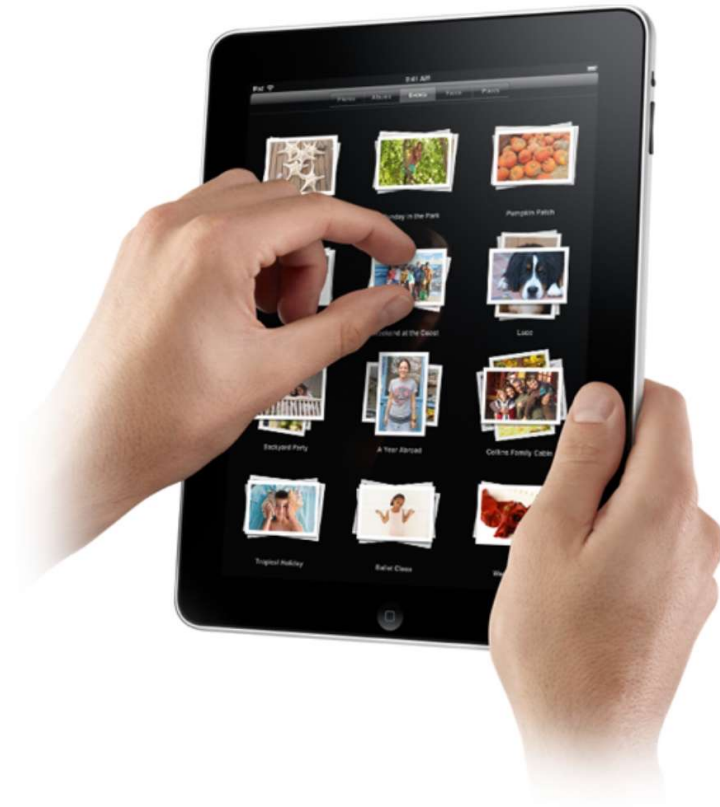
The BIG Picture



- Same components for all kinds of computer
 - Desktop, server, embedded
- Input/output includes
 - User-interface devices
 - Display, keyboard, mouse
 - Storage devices
 - Hard disk, CD/DVD, flash
 - Network adapters
 - For communicating with other computers

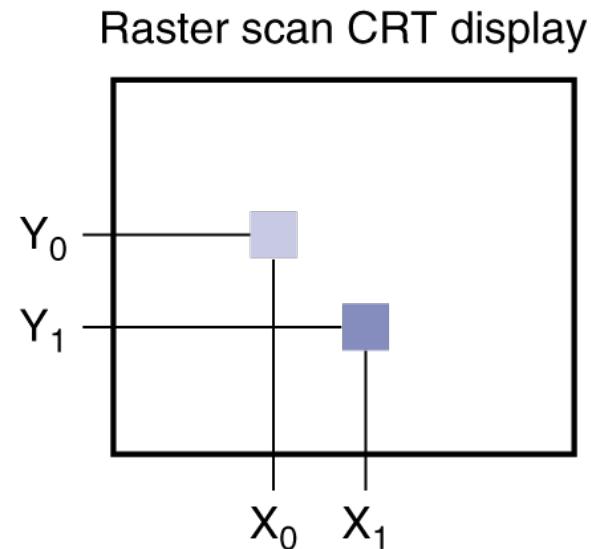
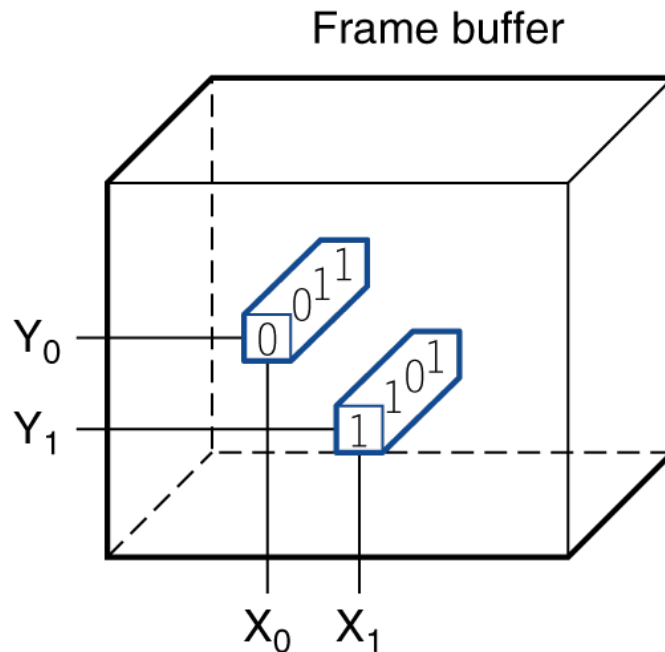
Touchscreen

- PostPC device
- Supersedes keyboard and mouse
- Resistive and Capacitive types
 - Most tablets, smart phones use capacitive
 - Capacitive allows multiple touches simultaneously

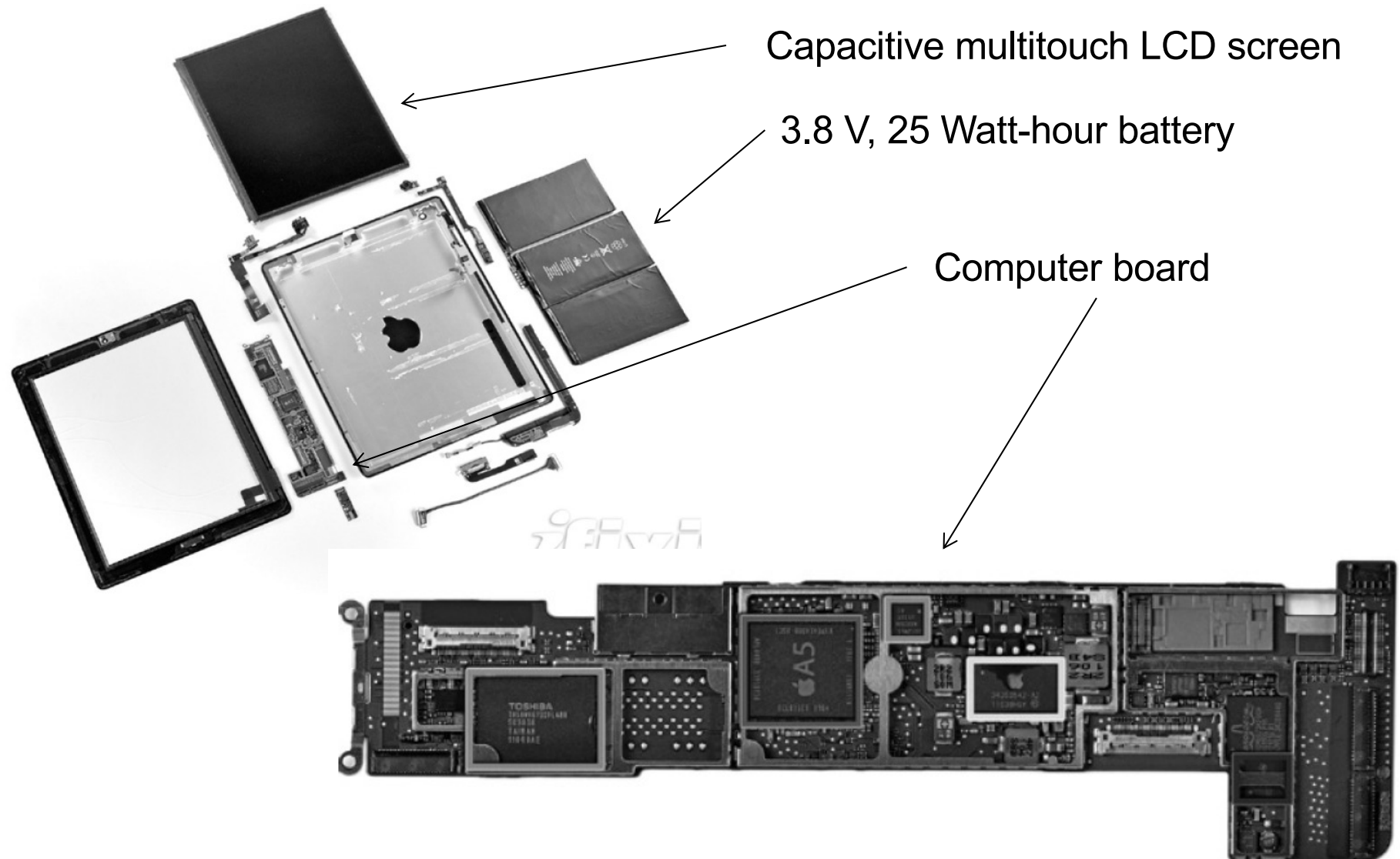


Through the Looking Glass

- LCD screen: picture elements (pixels)
 - Mirrors content of frame buffer memory



Opening the Box

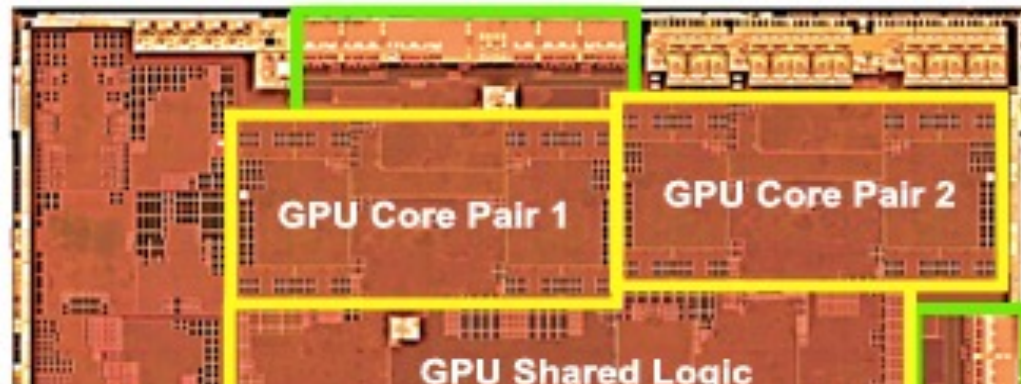


Inside the Processor (CPU)

- **Datapath**: performs operations on data
- **Control**: sequences datapath, memory, ...
- Cache memory
 - Small fast SRAM memory for immediate access to data

Inside the Processor

- Apple A9



One of AnandTech's more interesting findings is a significant reduction in the branch mispredict penalty, anywhere from 33 to 50 percent. It's not clear how this is achieved, but it's likely a combination of the larger cache, lower cache latency, and more aggressive prefetching. Because real-world code is generally branch heavy, this change could have a noticeable impact on user experience.



to the Typhoon core in the A8 SoC. Based on AnandTech's analysis, Twister has the same IPC (instructions per cycle) as Typhoon—capable of decoding, issuing, and executing up to six instructions per cycle—and retains a comparatively large out-of-order instruction reorder buffer. The execution stage sees a few minor tweaks, though, including a one cycle reduction in floating-point latency (a 20 percent improvement for multiplies and 25 percent for additions) and the ability to perform three FP32 floating-point multiply instructions in parallel instead of two.

Apple M1&M2



General information

Launched	M2: June 24, 2022 M2 Pro and Max: January 17, 2023 M2 Ultra: June 13, 2023 ^[1]
Designed by	Apple Inc.
Common manufacturer(s)	TSMC

Performance

Max. CPU clock rate	3.49 GHz ^[citation needed]
----------------------------	---------------------------------------

Cache

L1 cache	Performance cores 192+128 KB per core Efficiency cores 128+64 KB per core
-----------------	--

L2 cache	Performance cores M2: 16 MB M2 Pro and M2 Max: 32 MB M2 Ultra: 64 MB
-----------------	---

Efficiency cores

M2, M2 Pro, M2 Max: 4 MB M2 Ultra: 8 MB
--

Last level cache	M2: 8 MB M2 Pro: 24 MB M2 Max: 48 MB M2 Ultra: 96 MB
-------------------------	---

Apple M1&M2



Architecture and classification

Application	M2: Notebook (MacBook family), tablet (iPad Pro), desktop (Mac Mini), mixed reality headset (Vision Pro) ^[2] M2 Pro: Notebook (MacBook Pro), desktop (Mac mini) M2 Max: Notebook (MacBook Pro), desktop (Mac Studio) M2 Ultra: Desktop (Mac Studio, Mac Pro)
Technology node	5 nm (N5P)
Microarchitecture	"Avalanche" and "Blizzard"
Instruction set	ARMv8.5-A ^[citation needed]

Apple M1&M2



Physical specifications

Transistors

M2: 20 billion

M2 Pro: 40 billion

M2 Max: 67 billion

M2 Ultra: 134 billion

Cores

M2: 8 (4× high-performance + 4× high-efficiency)

M2 Pro: 10 or 12 (6× or 8× high-performance + 4× high-efficiency)

M2 Max: 12 (8× high-performance + 4× high-efficiency)

M2 Ultra: 24 (16× high-performance + 8× high-efficiency)

GPU(s)

Apple-designed integrated graphics

M2: 8 or 10 core GPU

M2 Pro: 16 or 19 core GPU

M2 Max: 30 or 38 core GPU

M2 Ultra: 60 or 76 core GPU

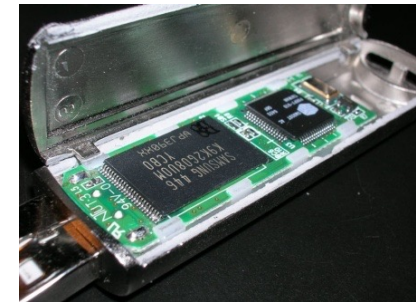
Abstractions

The BIG Picture

- Abstraction helps us deal with complexity
 - Hide lower-level detail
- Instruction set architecture (ISA)
 - The hardware/software interface
- Application binary interface
 - The ISA plus system software interface
- Implementation
 - The details underlying and interface

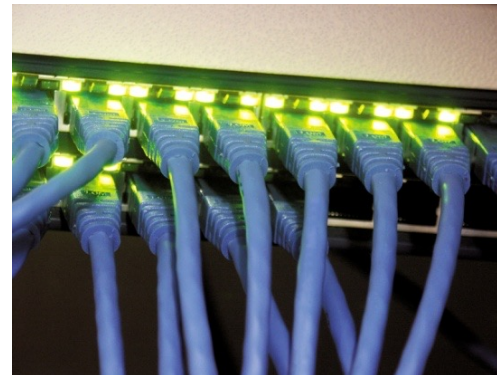
A Safe Place for Data

- Volatile main memory
 - Loses instructions and data when power off
- Non-volatile secondary memory
 - Magnetic disk
 - Flash memory
 - Optical disk (CDROM, DVD)



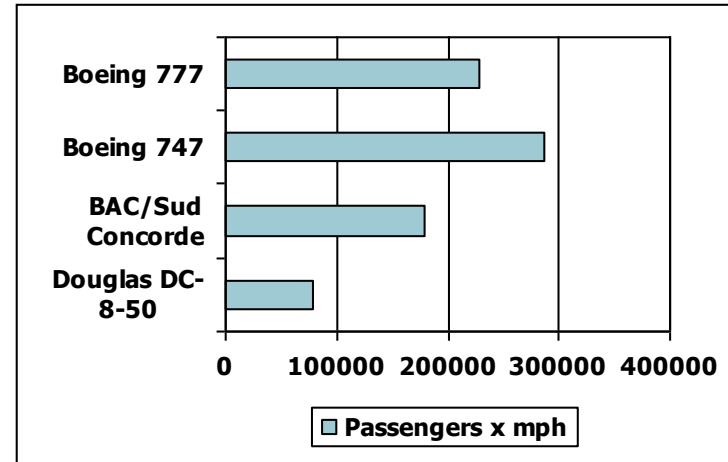
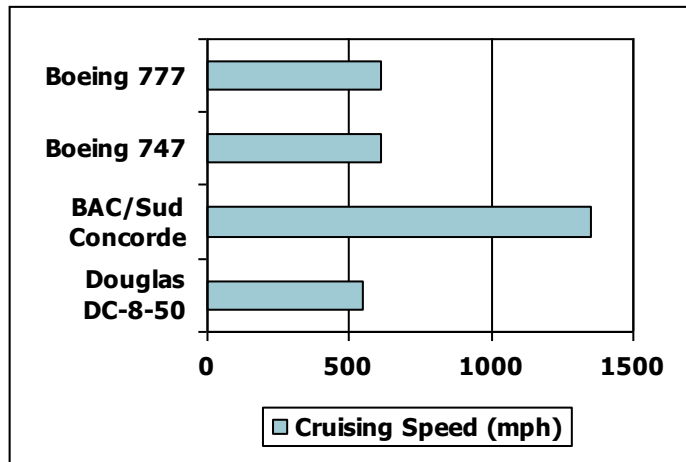
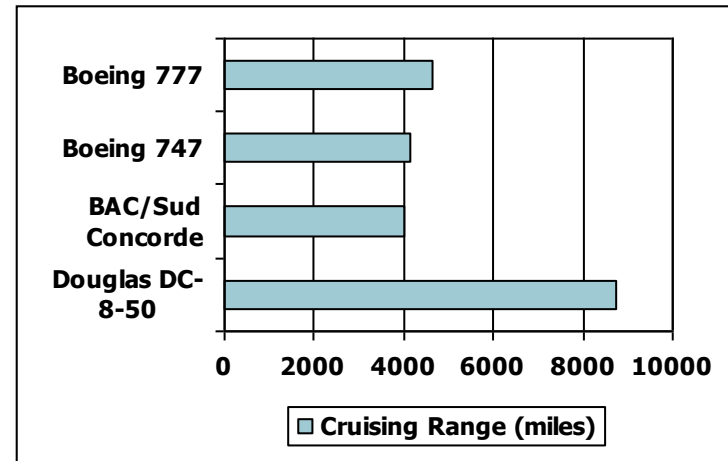
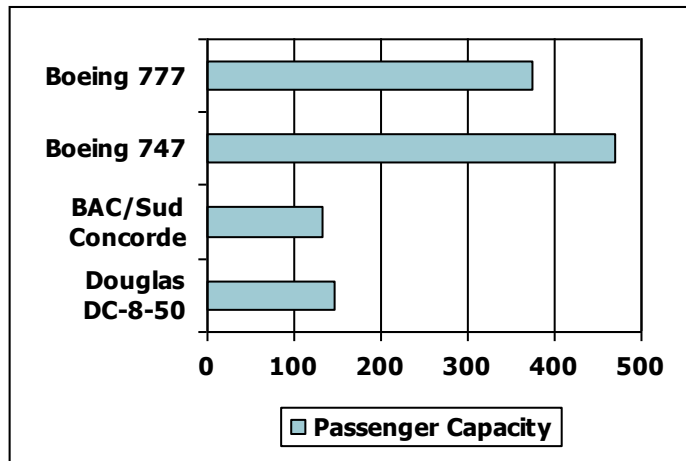
Networks

- Communication, resource sharing, nonlocal access
- Local area network (LAN): Ethernet
- Wide area network (WAN): the Internet
- Wireless network: WiFi, Bluetooth



Defining Performance

- Which airplane has the best performance?



Response Time and Throughput

- Response time
 - How long it takes to do a task
- Throughput
 - Total work done per unit time
 - e.g., tasks/transactions/... per hour
- How are response time and throughput affected by
 - Replacing the processor with a faster version?
 - Adding more processors?
- We'll focus on response time for now...

Relative Performance

- Define Performance = $1/\text{Execution Time}$
- “X is n time faster than Y”

$$\begin{aligned} & \text{Performance}_X / \text{Performance}_Y \\ &= \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

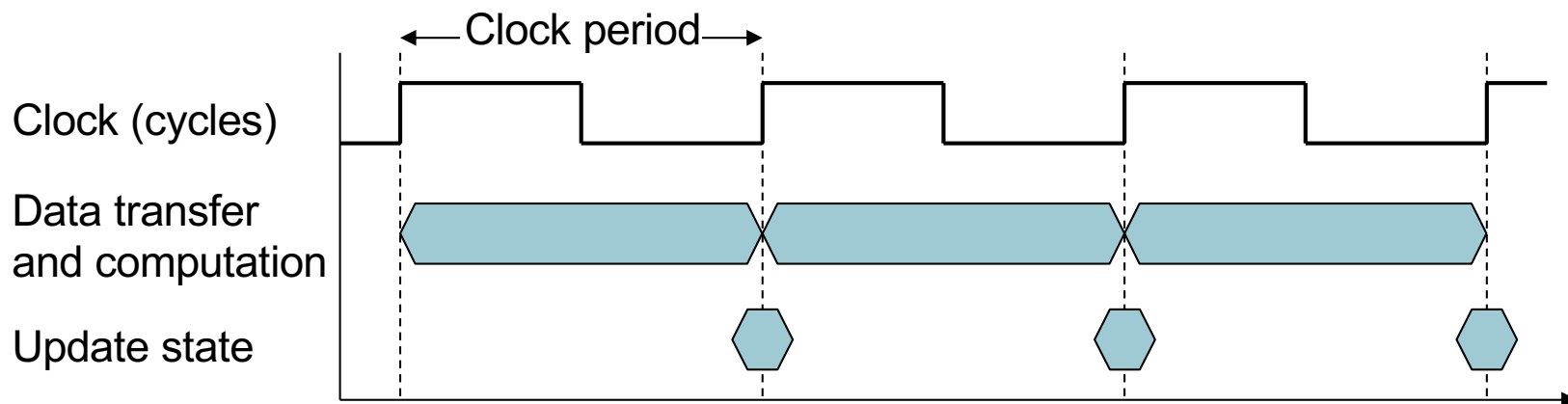
- Example: time taken to run a program
 - 10s on A, 15s on B
 - $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15\text{s} / 10\text{s} = 1.5$
 - So A is 1.5 times faster than B

Measuring Execution Time

- Elapsed time
 - Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
 - Determines system performance
- CPU time
 - Time spent processing a given job
 - Discounts I/O time, other jobs' shares
 - Comprises user CPU time and system CPU time
 - Different programs are affected differently by CPU and system performance

CPU Clocking

- Operation of digital hardware governed by a constant-rate clock



- Clock period: duration of a clock cycle
 - e.g., $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$
- Clock frequency (rate): cycles per second
 - e.g., $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$

CPU Time

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

- Performance improved by
 - Reducing number of clock cycles
 - Increasing clock rate
 - Hardware designer must often trade off clock rate against cycle count

CPU Time Example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
 - Aim for 6s CPU time
 - Can do faster clock, but causes $1.2 \times$ clock cycles
- How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$

Instruction Count and CPI

$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per Instruction}$

$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction Count for a program
 - Determined by program, ISA and compiler
- Average cycles per instruction
 - Determined by CPU hardware
 - If different instructions have different CPI
 - Average CPI affected by instruction mix

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster, and by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \leftarrow \text{A is faster...}\end{aligned}$$

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}\end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2 \leftarrow \text{...by this much}$$

CPI in More Detail

- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

- Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right)$$

Relative frequency

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5
 - Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
 - Avg. CPI = $10/5 = 2.0$

- Sequence 2: IC = 6
 - Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$
 - Avg. CPI = $9/6 = 1.5$

Performance Summary

The BIG Picture

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- Performance depends on
 - Algorithm: affects IC, possibly CPI
 - Programming language: affects IC, CPI
 - Compiler: affects IC, CPI
 - Instruction set architecture: affects IC, CPI, T_c

Pitfall: Amdahl's Law

- Improving an aspect of a computer and expecting a proportional improvement in overall performance

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- Example: multiply accounts for 80s/100s
 - How much improvement in multiply performance to get 5× overall?

$$20 = \frac{80}{n} + 20 \quad \text{■ Can't be done!}$$

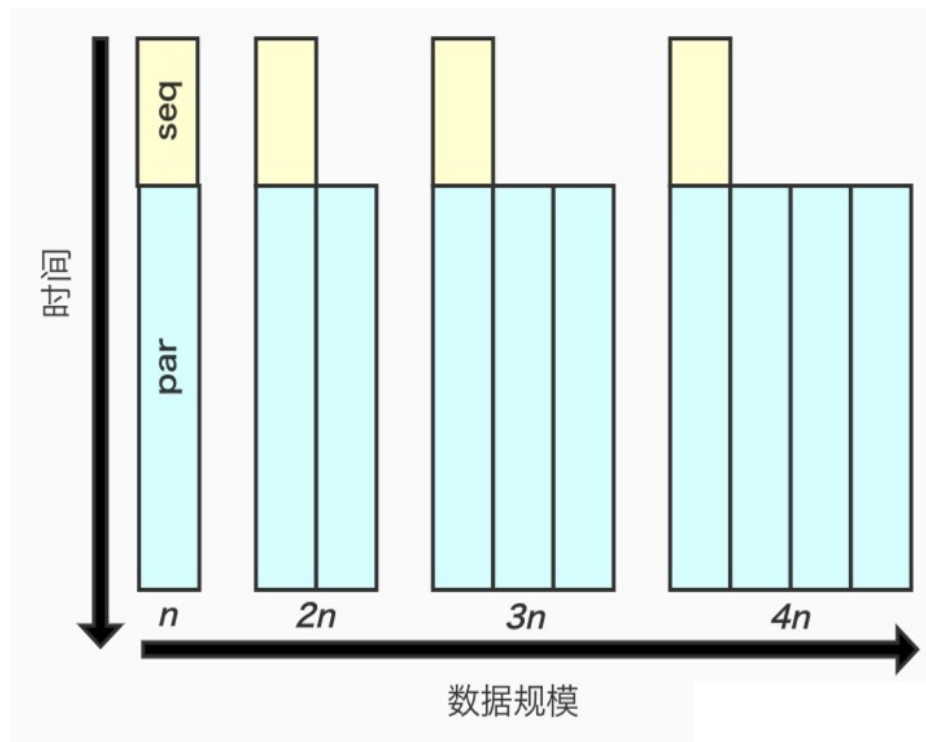
- Corollary: make the common case fast

Gustafson定律

- John L. Gustafson 在1988年提出的，是并行计算领域除了 Amdahl 定律之后又一个重要定律。

- Amdahl 定律有一个重要前提，就是处理的数据集大小是固定的，但是在大数据背景下，并行处理时，数据集大小是可变的（可增加的）

- Gustafson观测到在一个拥有1024个处理器的计算机，观察到了超线性加速比，分别获得了1021x/1020x/1016x的加速比



数据规模、处理器个数与时间的关系

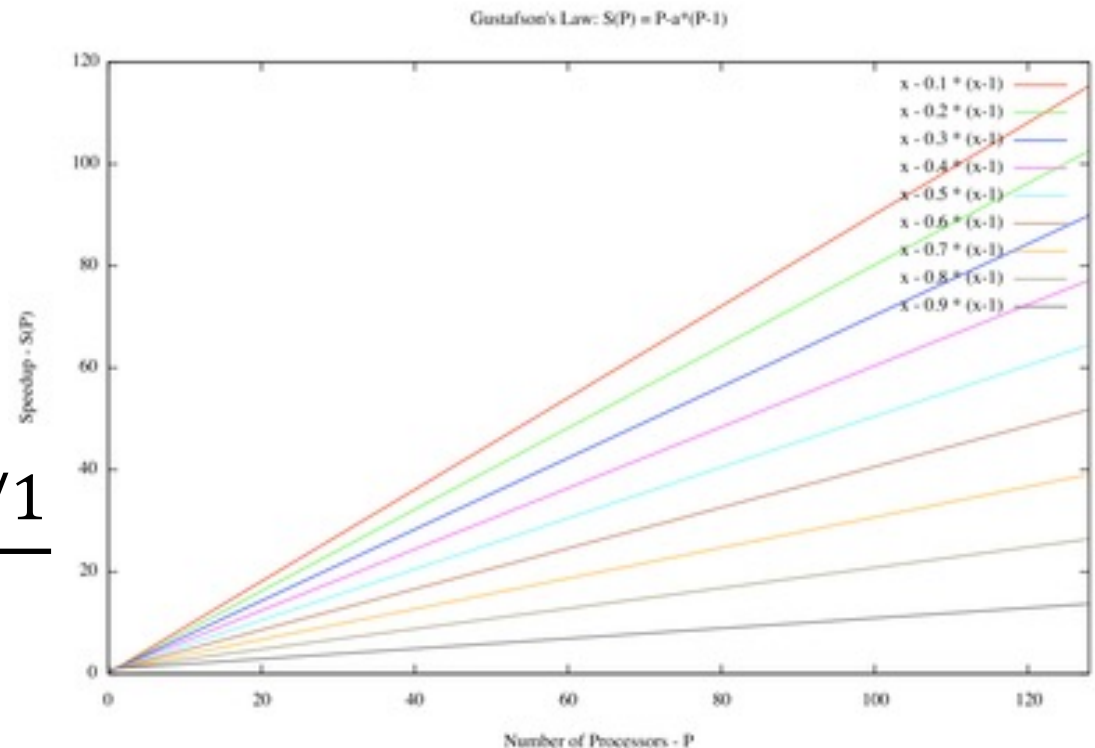
假设在 n 个节点上工作负载提升到

$$W' = \alpha W + (1 - \alpha)nW$$

由此，加速比

$$\begin{aligned} S'_n &= \frac{\text{单处理器执行时间}}{\text{n处理器执行时间}} \\ &= \frac{(\alpha W + (1 - \alpha)nW)/1}{\frac{\alpha W}{1} + \frac{(1 - \alpha)nW}{n}} \\ &= \alpha + (1 - \alpha)n \end{aligned}$$

α 为串行的比例



Fallacy: Low Power at Idle

- Look back at i7 power benchmark
 - At 100% load: 258W
 - At 50% load: 170W (66%)
 - At 10% load: 121W (47%)
- Google data center
 - Mostly operates at 10% – 50% load
 - At 100% load less than 1% of the time
- Consider designing processors to make power proportional to load

Pitfall: MIPS as a Performance Metric

- MIPS: Millions of Instructions Per Second
 - Doesn't account for
 - Differences in ISAs between computers
 - Differences in complexity between instructions

$$\begin{aligned}\text{MIPS} &= \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} \\ &= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}\end{aligned}$$

- CPI varies between programs on a given CPU

Concluding Remarks

- Cost/performance is improving
 - Due to underlying technology development
- Hierarchical layers of abstraction
 - In both hardware and software
- Instruction set architecture
 - The hardware/software interface
- Execution time: the best performance measure
- Power is a limiting factor
 - Use parallelism to improve performance

HW Chapter 1

在希冀平台查阅和提交

- 1.6
- 1.9
- 1.12
- 1.14